

P1152R4

Deprecating **volatile**

Published Proposal, 2019-07-19

This version:

<http://wg21.link/P1152R4>

Author:

[JF Bastien](#) (Apple)

Audience:

CWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/jfbastien/papers/blob/master/source/P1152R4.bs

Table of Contents

1	Abstract
2	Edit History
2.1	r3 → r4
2.2	r2 → r3
2.3	r1 → r2
2.4	r0 → r1
3	Wording
3.1	Program execution [intro.execution]
3.2	Data races [intro.races]
3.3	Forward progress [intro.progress]
3.4	Increment and decrement [expr.post.incr]
3.5	Class member access [expr.ref]

- 3.6 Increment and decrement [**expr.pre.incr**]
- 3.7 Assignment and compound assignment operators [**expr.ass**]
- 3.8 The cv-qualifiers [**dcl.type.cv**]
- 3.9 Functions [**dcl.fct**]
- 3.10 Structured binding declarations [**dcl.struct.bind**]
- 3.11 Non-static member functions [**class.mfct.non-static**]
- 3.12 The this pointer [**class.this**]
- 3.13 Constructors [**class.ctor**]
- 3.14 Destructors [**class.dtor**]
- 3.15 Overloadable declarations [**over.load**]
- 3.16 Built-in operators [**over.built**]
- 3.17 Annex D
 - 3.17.1 Deprecated **volatile** types [**depr.volatile.type**]

References

Informative References

§ 1. Abstract

We propose deprecating most of **volatile**. See [§ 3 Wording](#) for the details.

The proposed deprecation preserves the useful parts of **volatile**, and removes the dubious / already broken ones. This paper aims at breaking at compile-time code which is today subtly broken at runtime or through a compiler update. The paper might also break another type of code: that which doesn't exist. This removes a significant foot-gun and removes unintuitive corner cases from the languages.

The first version of this paper, [\[P1152R0\]](#), has extensive background information which is not repeated here:

- [Overview](#)
- [How we got here](#)
- [Why the proposed changes?](#)
- [Examples](#)

See [\[P1382R0\]](#) for the follow-up paper on **volatile_load<T>** / **volatile_store<T>** requested by SG1.

In Cologne, CWG was able to review this paper but LWG was not. The library parts of this paper have therefore moved to [\[P1831R0\]](#) so that the language changes can make C++20, and the library changes can be added to C++20 later.

§ 2. Edit History

§ 2.1. r3 → r4

Edit wording of the following sections as suggested by Jens Maurer: **[`expr.post.incr`]**, **[`expr.pre.incr`]**, **[`expr.ass`]**, **[`dcl.fct`]**, **[`over.load`]**, **[`over.built`]**.

Drop one word from **[`tuple`]** as suggested by Arthur O'Dwyer.

Add **[`dcl.struct.bind`]**.

Add Annex D.

Move library parts to [\[P1831R0\]](#).

§ 2.2. r2 → r3

[\[P1152R2\]](#) was reviewed offline by Alisdair Meredith.

- `is_volatile<T>` should have been `is_volatile_v<T>`.
- Use *Mandates* instead of *Constraints* to allow marking a function as deleted or employing a `static_assert` instead of using `enable_if` or `requires`. This likely provides better error messages.
- "`is_volatile_v<T>` is `false`" was erroneously used instead of testing whether the `this` pointer itself was `volatile`. Wording of the *Mandates* clause was updated to only apply to the `volatile` overloads of methods instead.

§ 2.3. r1 → r2

[\[P1152R1\]](#) was seen by SG1 and EWG in Kona. This update does the following:

- Also edit sections **[`expr.post.incr`]**, **[`expr.pre.incr`]**, **[`expr.ass`]**, which are redundant with other sections already modified by this paper.

- Change *Remarks* to *Constraints* per [\[P1369R0\]](#).
- Change language wording to explicitly call out deprecation.

Poll	Group	SF	F	N	A	SA	Outcome
Forward this paper—with edits as discussed —to EWG for C++20.	SG1	3	12	1	1	0	✓
Proposal as presented for C++20.	EWG	6	27	1	0	0	✓

§ 2.4. r0 → r1

[\[P1152R0\]](#) was seen by SG1 and EWG in San Diego. This update does the following:

- Remove background information from the paper.
- Follow the guidance from SG1 and EWG, based on the polls below.

Poll	Group	SF	F	N	A	SA	Outcome
Deprecate volatile compound operations (including ++ and --) on scalar types (arithmetic, pointer, enumeration).	SG1	4	19	3	0	0	✓
Deprecate volatile compound operations (including ++ and --) on scalar types (arithmetic, pointer, enumeration).	EWG	4	9	4	0	0	✓
Deprecate usage of volatile assignment chaining on scalar types (arithmetic, pointer, enumeration, pointer to members, <code>nullptr_t</code>).	SG1	6	15	3	0	0	✓
Deprecate usage of volatile assignment chaining on scalar types (arithmetic, pointer, enumeration, pointer to members, <code>nullptr_t</code>).	EWG	6	9	3	0	0	✓
SG1 would be OK if we deprecated volatile -qualified member functions (pending separate decision on what we do with volatile atomic).	SG1	1	5	10	4	3	✗
EWG would be OK if we deprecated volatile -qualified member functions (pending separate decision on what we do with volatile atomic).	EWG	2	7	7	1	0	✓

Poll	Group	SF	F	N	A	SA	Outcome
SG1 would be OK if we deprecated volatile partial template specializations, overloads, or qualified member functions in the STL for all but the atomic, numeric_limits , and type traits (remove_volatile , add_volatile , etc) parts of the Library.	SG1	1	9	6	2	0	✓
EWG would be OK if we deprecated volatile partial template specializations, overloads, or qualified member functions in the STL for all but the atomic, numeric_limits , and type traits (remove_volatile , add_volatile , etc) parts of the Library.	EWG	1	11	9	0	0	✓
Deprecate volatile member functions of atomic in favor of new template partial specializations which will only declare load, store, and only exist when is_always_lock_free is true.	SG1	2	1	1	11	2	✗
Deprecate volatile member functions of atomic in favor of new template partial specializations which will only declare load, store, RMW, and only exist when is_always_lock_free is true.	SG1	4	7	3	3	0	✓
Deprecate volatile member functions of atomic in favor of new template partial specializations which will only declare load, store, RMW, and only exist when is_always_lock_free is true.	EWG	2	9	3	0	0	✓
Deprecate volatile member functions of atomic in favor of new template partial specializations which will only declare load, store, RMW.	SG1	0	0	0	10	7	✗
SG1 would be OK if we deprecated top-level volatile parameters.	SG1	6	9	6	2	1	✓
EWG would be OK if we deprecated top-level volatile parameters.	EWG	6	9	6	0	0	✓

Poll	Group	SF	F	N	A	SA	Outcome
EWG would be OK if we deprecated top-level const parameters.	EWG	0	2	5	8	8	✗
SG1 would be OK if we deprecated top-level <code>volatile</code> return values.	SG1	6	9	4	2	0	✓
EWG would be OK if we deprecated top-level <code>volatile</code> return values.	EWG	6	6	5	0	0	✓
EWG would be OK if we deprecated top-level const return values.	EWG	2	3	3	5	5	✗
SG1 interested is interested in hearing about <code>volatile_load<T></code> / <code>volatile_store<T></code> free functions in a separate paper, given that time is limited and we could be doing something else.	SG1	0	17	4	3	0	✓
EWG interested is interested in hearing about <code>volatile_load<T></code> / <code>volatile_store<T></code> free functions in a separate paper, given that time is limited and we could be doing something else.	EWG	2	11	4	1	0	✓

§ 3. Wording

The proposed wording follows the language and library approach to deprecation:

- Language deprecation is called out in the Standard text itself, and repeated in Annex D.
- Library deprecation presents the library without the deprecated feature, and only mentions said feature in Annex D.

§ 3.1. Program execution [[intro.execution](#)]

No changes.

Accesses through `volatile` glvalues are evaluated strictly according to the rules of the abstract machine.

Reading an object designated by a `volatile` glvalue, modifying an object, calling a library I/O function, or calling a function that does any of those operations are all *side effects*, which are changes in the state of the execution environment. *Evaluation* of an expression (or a subexpression) in general includes both value computations (including determining the identity of an object for glvalue evaluation and fetching a value previously assigned to an object for prvalue evaluation) and initiation of side effects. When a call to a library I/O function returns or an access through a `volatile` glvalue is evaluated the side effect is considered complete, even though some external actions implied by the call (such as the I/O itself) or by the `volatile` access may not have completed yet.

§ 3.2. Data races [intro.races]

No changes.

Two accesses to the same object of type `volatile std::sig_atomic_t` do not result in a data race if both occur in the same thread, even if one or more occurs in a signal handler. For each signal handler invocation, evaluations performed by the thread invoking a signal handler can be divided into two groups A and B, such that no evaluations in B happen before evaluations in A, and the evaluations of such `volatile std::sig_atomic_t` objects take values as though all evaluations in A happened before the execution of the signal handler and the execution of the signal handler happened before all evaluations in B.

§ 3.3. Forward progress [intro.progress]

No changes.

The implementation may assume that any thread will eventually do one of the following:

- terminate,
- make a call to a library I/O function,
- perform an access through a `volatile` glvalue, or
- perform a synchronization operation or an atomic operation

During the execution of a thread of execution, each of the following is termed an execution step:

- termination of the thread of execution,
- performing an access through a `volatile` glvalue, or
- completion of a call to a library I/O function, a synchronization operation, or an atomic operation.

§ 3.4. Increment and decrement [`expr.post.incr`]

Modify as follows.

The value of a postfix `++` expression is the value of its operand. [*Note*: The value obtained is a copy of the original value —*end note*] The operand shall be a modifiable lvalue. The type of the operand shall be an arithmetic type other than `cv bool`, or a pointer to a complete object type. An operand with `volatile`-qualified type is deprecated; see [`depr.volatile.type`]. The value of the operand object is modified by adding 1 to it. The value computation of the `++` expression is sequenced before the modification of the operand object. With respect to an indeterminately-sequenced function call, the operation of postfix `++` is a single evaluation. [*Note*: Therefore, a function call shall not intervene between the lvalue-to-rvalue conversion and the side effect associated with any single postfix `++` operator. —*end note*] The result is a prvalue. The type of the result is the `cv`-unqualified version of the type of the operand. If the operand is a bit-field that cannot represent the incremented value, the resulting value of the bit-field is implementation-defined. See also [`expr.add`] and [`expr.ass`].

The operand of postfix `--` is decremented analogously to the postfix `++` operator. [*Note*: For prefix increment and decrement, see [`expr.pre.incr`]. —*end note*]

§ 3.5. Class member access [**expr.ref**]

No changes.

Abbreviating *postfix-expression.id-expression* as $E1.E2$, $E1$ is called the *object expression*. If $E2$ is a bit-field, $E1.E2$ is a bit-field. The type and value category of $E1.E2$ are determined as follows. In the remainder of [**expr.ref**], *cq* represents either `const` or the absence of `const` and *vq* represents either `volatile` or the absence of `volatile`. *cv* represents an arbitrary set of cv-qualifiers.

- If $E2$ is a non-static data member and the type of $E1$ is “*cq1 vq1 X*”, and the type of $E2$ is “*cq2 vq2 T*”, the expression designates the named member of the object designated by the first expression. If $E1$ is an lvalue, then $E1.E2$ is an lvalue; otherwise $E1.E2$ is an xvalue. Let the notation *vq12* stand for the “union” of *vq1* and *vq2*; that is, if *vq1* or *vq2* is `volatile`, then *vq12* is `volatile`. Similarly, let the notation *cq12* stand for the “union” of *cq1* and *cq2*; that is, if *cq1* or *cq2* is `const`, then *cq12* is `const`. If $E2$ is declared to be a `mutable` member, then the type of $E1.E2$ is “*vq12 T*”. If $E2$ is not declared to be a `mutable` member, then the type of $E1.E2$ is “*cq12 vq12 T*”.

§ 3.6. Increment and decrement [**expr.pre.incr**]

Modify as follows.

The operand of prefix `++` is modified by adding 1. The operand shall be a modifiable lvalue. The type of the operand shall be an arithmetic type other than `cv bool`, or a pointer to a completely-defined object type. An operand with `volatile`-qualified type is deprecated; see [**depr.volatile.type**]. The result is the updated operand; it is an lvalue, and it is a bit-field if the operand is a bit-field. The expression `++x` is equivalent to `x+=1`. [*Note*: See the discussions of [**expr.add**] and assignment operators [**expr.ass**] for information on conversions. —*end note*]

The operand of prefix `--` is modified by subtracting 1. The requirements on the operand of prefix `--` and the properties of its result are otherwise the same as those of prefix `++`. [*Note*: For postfix increment and decrement, see [**expr.post.incr**]. —*end note*]

§ 3.7. Assignment and compound assignment operators [**expr.ass**]

Modify as follows.

1. The assignment operator (=) and the compound assignment operators all group right-to-left.
2. All require a modifiable lvalue as their left operand; their result is an lvalue referring to the left operand. The result in all cases is a bit-field if the left operand is a bit-field. In all cases, the assignment is sequenced after the value computation of the right and left operands, and before the value computation of the assignment expression. The right operand is sequenced before the left operand. With respect to an indeterminately-sequenced function call, the operation of a compound assignment is a single evaluation. [*Note*: Therefore, a function call shall not intervene between the lvalue-to-rvalue conversion and the side effect associated with any single compound assignment operator. —*end note*]

assignment-expression

conditional-expression

logical-or-expression assignment-operator initializer-clause

throw-expression

assignment-operator: one of

= *= /= %= += -= >>= <<= &= ^= |=

3. In simple assignment (=), the object referred to by the left operand is modified by replacing its value with the result of the right operand.
4. If the left operand is not of class type, the expression is implicitly converted to the cv-qualified type of the left operand.
5. If the left operand is of class type, the class shall be complete. Assignment to objects of a class is defined by the copy/move assignment.
6. [*Note*: For class objects, assignment is not in general the same as initialization. —*end note*]
7. When the left operand of an assignment operator is a bit-field that cannot represent the value of the expression, the resulting value of the bit-field is implementation-defined.
8. Simple assignments where the left operand is a volatile-qualified type that is not of class type are deprecated (see [depr.volatile.type]) unless they are either a discarded-value expression or appear in an unevaluated context.
9. The behavior of an expression of the form $E1 \text{ op} = E2$ is equivalent to $E1 = E1 \text{ op} E2$ except that $E1$ is evaluated only once. Such expressions are deprecated if $E1$ has volatile-qualified type; see [depr.volatile.type]. In += and -=, $E1$ shall either have arithmetic type or be a pointer to a possibly cv-qualified completely-defined object type. In all other cases, $E1$ shall have arithmetic type.

10. If the value being stored in an object is read via another object that overlaps in any way the storage of the first object, then the overlap shall be exact and the two objects shall have the same type, otherwise the behavior is undefined. [*Note:* This restriction applies to the relationship between the left and right sides of the assignment operation; it is not a statement about how the target of the assignment may be aliased in general. See [basic.lval]. —end note]
11. A *braced-init-list* may appear on the right-hand side of
 1. an assignment to a scalar, in which case the initializer list shall have at most a single element. The meaning of $x = \{v\}$, where T is the scalar type of the expression x , is that of $x = T\{v\}$. The meaning of $x = \{\}$ is $x = T\{\}$.
 2. an assignment to an object of class type, in which case the initializer list is passed as the argument to the assignment operator function selected by overload resolution.

§ 3.8. The cv-qualifiers [dcl.type.cv]

No changes.

The semantics of an access through a `volatile` glvalue are implementation-defined. If an attempt is made to access an object defined with a `volatile`-qualified type through the use of a non-`volatile` glvalue, the behavior is undefined.

[*Note:* `volatile` is a hint to the implementation to avoid aggressive optimization involving the object because the value of the object might be changed by means undetectable by an implementation. Furthermore, for some implementations, `volatile` might indicate that special hardware instructions are required to access the object. See [intro.execution] for detailed semantics. In general, the semantics of `volatile` are intended to be the same in C++ as they are in C. —end note]

§ 3.9. Functions [dcl.fct]

Modify as follows.

The *parameter-declaration-clause* determines the arguments that can be specified, and their processing, when the function is called. [*Note*: The *parameter-declaration-clause* is used to convert the arguments specified on the function call; see [expr.call] —end note] If the *parameter-declaration-clause* is empty, the function takes no arguments. A parameter list consisting of a single unnamed parameter of non-dependent type `void` is equivalent to an empty parameter list. Except for this special case, a parameter shall not have type `cv void`. A parameter with `volatile-qualified` type is deprecated; see [depr.volatile.type]. If the *parameter-declaration-clause* terminates with an ellipsis or a function parameter pack, the number of arguments shall be equal to or greater than the number of parameters that do not have a default argument and are not function parameter packs. Where syntactically correct and where "`...`" is not part of an *abstract-declarator*, "`, ...`" is synonymous with "`...`".

[...]

The type of a function is determined using the following rules. The type of each parameter (including function parameter packs) is determined from its own *decl-specifier-seq* and *declarator*. After determining the type of each parameter, any parameter of type "array of T" or of function type T is adjusted to be "pointer to T". After producing the list of parameter types, any top-level *cv-qualifiers* modifying a parameter type are deleted when forming the function type. The resulting list of transformed parameter types and the presence or absence of the ellipsis or a function parameter pack is the function's *parameter-type-list*.

[...]

Functions shall not have a return type of type array or function, although they may have a return type of type pointer or reference to such things. There shall be no arrays of functions, although there can be arrays of pointers to functions.

A `volatile-qualified` return type is deprecated; see [depr.volatile.type].

§ 3.10. Structured binding declarations [dcl.struct.bind]

Modify as follows:

A structured binding declaration introduces the *identifiers* $v_0, v_1, v_2, \dots$ of the *identifier-list* as names of *structured bindings*. Let *cv* denote the *cv-qualifiers* in the *decl-specifier-seq* and *S* consist of the *storage-class-specifiers* of the *decl-specifier-seq* (if any). A *cv* that includes *volatile* is deprecated; see [depr.volatile.type]. First, a variable with a unique name *e* is introduced. If the *assignment-expression* in the *initializer* has array type *A* and no *ref-qualifier* is present, *e* is defined by

*attribute-specifier-seq*_{opt} *S cv A e* ;

§ 3.11. Non-static member functions [class.mfct.non-static]

No changes.

A non-static member function may be declared `const`, `volatile`, or `const volatile`. These *cv-qualifiers* affect the type of the `this` pointer. They also affect the function type of the member function; a member function declared `const` is a `const` member function, a member function declared `volatile` is a `volatile` member function and a member function declared `const volatile` is a `const volatile` member function.

§ 3.12. The `this` pointer [class.this]

No changes.

In the body of a non-static member function, the keyword `this` is a prvalue expression whose value is the address of the object for which the function is called. The type of `this` in a member function of a class *X* is *X**. If the member function is declared `const`, the type of `this` is `const X*`, if the member function is declared `volatile`, the type of `this` is `volatile X*`, and if the member function is declared `const volatile`, the type of `this` is `const volatile X*`.

`volatile` semantics apply in `volatile` member functions when accessing the object and its non-static data members.

§ 3.13. Constructors [class.ctor]

No changes.

A constructor can be invoked for a `const`, `volatile` or `const volatile` object. `const` and `volatile` semantics are not applied on an object under construction. They come into effect when the constructor for the most derived object ends.

§ 3.14. Destructors [**class.dtor**]

No changes.

A destructor is used to destroy objects of its class type. The address of a destructor shall not be taken. A destructor can be invoked for a `const`, `volatile` or `const volatile` object. `const` and `volatile` semantics are not applied on an object under destruction. They stop being in effect when the destructor for the most derived object starts.

§ 3.15. Overloadable declarations [**over.load**]

No changes since [**dcl.fct**] already handles deprecation.

Parameter declarations that differ only in the presence or absence of `const` and/or `volatile` are equivalent. That is, the `const` and `volatile` type-specifiers for each parameter type are ignored when determining which function is being declared, defined, or called.

§ 3.16. Built-in operators [**over.built**]

No changes since [**expr.post.incr**], [**expr.pre.incr**], and [**expr.ass**] already handle deprecation.

In the remainder of this section, *vk* represents either `volatile` or no cv-qualifier.

For every pair (*T*, *vk*), where *T* is an arithmetic type other than `bool`, there exist candidate operator functions of the form

```
vk T & operator++(vk T &);  
T operator++(vk T &, int);
```

For every pair (*T*, *vk*), where *T* is an arithmetic type other than `bool`, there exist candidate operator functions of the form

```
vk T & operator--(vk T &);  
T operator--(vk T &, int);
```

For every pair (*T*, *vk*), where *T* is a cv-qualified or cv-unqualified object type, there exist candidate operator functions of the form

```
T*vk& operator++(T*vk&);  
T*vk& operator--(T*vk&);  
T* operator++(T*vk&, int);  
T* operator--(T*vk&, int);
```

For every quintuple (*C1*, *C2*, *T*, *cv1*, *cv2*), where *C2* is a class type, *C1* is the same type as *C2* or is a derived class of *C2*, and *T* is an object type or a function type, there exist candidate operator functions of the form

```
cv12 T& operator->*(cv1 C1*, cv2 T C2::*);
```

For every triple (*L*, *vk*, *R*), where *L* is an arithmetic type, and *R* is a promoted arithmetic type, there exist candidate operator functions of the form

```
vk L& operator=(vk L&, R);  
vk L& operator*=(vk L&, R);  
vk L& operator/=(vk L&, R);  
vk L& operator+=(vk L&, R);  
vk L& operator-=(vk L&, R);
```

For every pair (*T*, *vk*), where *T* is any type, there exist candidate operator functions of the form

```
T*vk& operator=(T*vk&, T*);
```

For every pair (T, vq) , where T is an enumeration or pointer to member type, there exist candidate operator functions of the form

```
vq T& operator=(vq T&, T );
```

For every pair (T, vq) , where T is a cv-qualified or cv-unqualified object type, there exist candidate operator functions of the form

```
T*vq& operator+=(T*vq&, std::ptrdiff_t);  
T*vq& operator-=(T*vq&, std::ptrdiff_t);
```

For every triple (L, vq, R) , where L is an integral type, and R is a promoted integral type, there exist candidate operator functions of the form

```
vq L& operator%=(vq L&, R);  
vq L& operator<<=(vq L&, R);  
vq L& operator>=(vq L&, R);  
vq L& operator&=(vq L&, R);  
vq L& operator^=(vq L&, R);  
vq L& operator|=(vq L&, R);
```

§ 3.17. Annex D

Add the following wording to Annex D:

§ 3.17.1. Deprecated **volatile** types [depr.volatile.type]

Postfix **++** and **--** expressions ([**expr.post.incr**]) and prefix **++** and **--** expressions ([**expr.pre.incr**]) of **volatile**-qualified arithmetic and pointer types are deprecated.

Certain assignments where the left operand is a **volatile**-qualified non-class type are deprecated; see [**expr.ass**].

A function type ([**dcl.fct**]) with a parameter with **volatile**-qualified type or with a **volatile**-qualified return type is deprecated.

A structured binding ([**dcl.struct.bind**]) of a **volatile**-qualified type is deprecated.

§ References

§ Informative References

[P1152R0]

JF Bastien. [Deprecating volatile](https://wg21.link/p1152r0). 1 October 2018. URL: <https://wg21.link/p1152r0>

[P1152R1]

JF Bastien. [Deprecating volatile](https://wg21.link/p1152r1). 20 January 2019. URL: <https://wg21.link/p1152r1>

[P1152R2]

JF Bastien. [Deprecating volatile](https://wg21.link/p1152r2). 9 March 2019. URL: <https://wg21.link/p1152r2>

[P1369R0]

Walter E. Brown. [Guidelines for Formulating Library Semantics Specifications](https://wg21.link/p1369r0). 25 November 2018. URL: <https://wg21.link/p1369r0>

[P1382R0]

JF Bastien, Paul McKenney. [volatile_load<T> and volatile_store<T>](https://wg21.link/p1382r0). 11 January 2019. URL: <https://wg21.link/p1382r0>

[P1831R0]

JF Bastien. [Deprecating volatile: library](https://wg21.link/P1831R0). 19 July 2019. URL: <https://wg21.link/P1831R0>